

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração - Aula 6

Spring Boot, Spring Cloud, Eureka, Zuul, Service Discovery e API Gateway

Material autoral - padrão FGV - sem flashcards - versão de estudo

Recorte da aula: Esta aula cobre a Aula 6 da trilha: Spring Boot, Spring Cloud, Eureka, Zuul, Service Discovery e API Gateway. No edital, o foco está nos itens 1.9 e 1.10 da disciplina Arquitetura de Sistemas e Integração.

Estratégia de prova: A FGV tende a cobrar distinções de função e escopo: framework versus ecossistema, configuração automática versus ausência de configuração, service discovery versus API Gateway, Eureka versus Zuul, roteamento versus balanceamento, e gateway como ponto de políticas transversais sem virar depósito de regra negocial.

1. Identificação da aula e aderência ao edital

Aula 6 da trilha: Spring Boot, Spring Cloud, Eureka, Zuul, Service Discovery e API Gateway.

Tópicos do edital cobertos:

- 1.9 Framework Spring Boot, Spring Cloud, Spring Eureka e Zuul.
- 1.10 Service Discovery e API Gateway.
- Conexões necessárias com 1.4 Microsserviços, 1.5 Integração de sistemas e 1.17 Ambientes distribuídos e escaláveis, apenas na medida em que explicam o papel dos componentes desta aula.

Objetivo: fazer você diferenciar, em cenário de prova, o que cada peça faz em uma arquitetura Java distribuída: Spring Boot cria aplicações Spring executáveis e configuráveis; Spring Cloud reúne padrões para sistemas distribuídos; Eureka registra e descobre serviços; Zuul atua como roteador/gateway legado; API Gateway centraliza entrada, roteamento e preocupações transversais; Service Discovery resolve dinamicamente onde estão as instâncias dos serviços.

2. Vocabulário essencial antes da teoria

Termo	Expansão / definição objetiva	Pegadinha FGV
API	Application Programming Interface. Interface contratual para que sistemas se comuniquem.	API não é necessariamente REST; pode ser SOAP, gRPC, biblioteca, evento, etc.
Endpoint	Ponto de acesso exposto por uma API, como GET /processos/{id}.	Endpoint não é o serviço inteiro; é uma operação/recurso exposto.
HTTP	Hypertext Transfer Protocol. Protocolo de aplicação usado na web.	HTTP é stateless por natureza; estado de sessão é construído por mecanismos adicionais.
TLS	Transport Layer Security. Camada de segurança para criptografia e autenticação no transporte.	HTTPS é HTTP sobre TLS; não corrige sozinho falhas de autorização da aplicação.
DNS	Domain Name System. Sistema de nomes que resolve nomes para endereços.	DNS ajuda localização por nome, mas não substitui necessariamente um registro dinâmico de serviços.
REST	Representational State Transfer. Estilo arquitetural orientado a recursos e representações.	REST não é sinônimo de JSON, HTTP ou Spring.
JSON	JavaScript Object Notation. Formato textual de representação de dados.	JSON não implica REST nem mensageria.
JVM	Java Virtual Machine. Ambiente de execução de bytecode Java.	Spring Boot e Spring Cloud rodam no ecossistema JVM, mas isso não define a arquitetura por si só.
Gateway	Componente de entrada que roteia chamadas e aplica preocupações transversais.	Gateway não é service discovery, nem banco de regras de negócio.
Service Discovery	Mecanismo para localizar dinamicamente instâncias de serviços.	Descobrir serviço não é rotear requisição pública nem autenticar usuário.

3. Spring Boot: o que é e o que não é

Spring Boot é uma forma opinativa de criar aplicações baseadas no ecossistema Spring com menos configuração manual. A documentação oficial o apresenta como ferramenta para criar aplicações Spring stand-alone e production-grade, executáveis com java -jar ou empacotadas em modelos tradicionais. Para prova, a essência é: **reduzir fricção de inicialização, empacotamento e configuração operacional, não abolir arquitetura.**

3.1 Problema que o Spring Boot resolve

Antes do Spring Boot, era comum perder muito tempo combinando versões, configurando servidor de aplicação, arquivos XML, beans, dependências e infraestrutura básica. Spring Boot **reduz essa fricção com convenções, starters, servidor embutido, configuração externa e auto-configuração**. Ele não decide sozinho suas fronteiras de domínio, sua estratégia de integração ou sua decomposição em serviços.

3.2 Elementos cobrados em prova

Elemento	Função	Como a FGV pode distorcer
Starter	Dependência agregadora que traz bibliotecas típicas de uma finalidade, como web, data-jpa ou security.	Dizer que starter é servidor, container Docker ou gateway.
Auto-configuração	Tenta configurar automaticamente a aplicação com base nas dependências presentes no classpath e nas propriedades.	Dizer que elimina toda configuração ou que sempre acerta sem intervenção.

@SpringBootApplication	Anotação composta usada como ponto de entrada de configuração da aplicação.	Dizer que transforma automaticamente monólito em microsserviço.
Embedded server	Permite rodar aplicação web com servidor embutido, como Tomcat, Jetty ou Netty, conforme stack.	Dizer que dispensa deploy, observabilidade, segurança ou infraestrutura.
Externalized configuration	Permite separar código de configuração de ambiente: application.yml, variáveis de ambiente, argumentos, etc.	Dizer que configuração externa é service discovery ou API Gateway.
Actuator	Fornece endpoints de operação, como health, info e métricas, quando configurado.	Dizer que Actuator é balanceador ou substitui monitoramento corporativo.

3.3 Exemplo prático ligado ao seu contexto

Imagine uma API Java para consulta de processos administrativos. Em Quarkus, você provavelmente pensaria em Resource, Service, Repository e application.properties. Em Spring Boot, a lógica de camadas pode ser parecida: Controller, Service, Repository e application.yml. A diferença principal, para esta aula, não é a regra de negócio: é o ecossistema Spring Boot/Spring Cloud para inicialização, configuração, integração, descoberta e roteamento em ambiente distribuído.

```
@SpringBootApplication
public class ProcessoApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProcessoApplication.class, args);
    }
}
```

```
@RestController
@RequestMapping("/processos")
class ProcessoController {
    private final ProcessoService service;

    @GetMapping("/{id}")
    ProcessoDTO consultar(@PathVariable Long id) {
        return service.consultar(id);
    }
}
```

Ponto de prova: Spring Boot facilita criação e operação de aplicações Spring. Ele não é, sozinho, arquitetura de microsserviços, API Gateway, Service Discovery ou plataforma de nuvem.

4. Spring Cloud: ecossistema para padrões de sistemas distribuídos

Spring Cloud **é um conjunto de projetos que ajuda a implementar padrões comuns em sistemas distribuídos**. O edital menciona Spring Cloud junto com Eureka, Zuul, Service Discovery e API Gateway porque esses componentes aparecem quando várias aplicações precisam se localizar, rotear chamadas, aplicar políticas transversais e tolerar instabilidade de rede.

4.1 Spring Boot versus Spring Cloud

Comparação	Spring Boot	Spring Cloud
Papel central	Criar e executar aplicações Spring com convenções e auto-configuração.	Implementar padrões de sistemas distribuídos sobre aplicações Spring Boot.
Escopo típico	Uma aplicação individual: API, job, serviço web.	Conjunto de aplicações que precisam de configuração, descoberta, roteamento e integração.
Problema resolvido	Reduzir configuração manual e facilitar produção.	Lidar com complexidade de ambientes distribuídos.
Exemplo	API de consulta processual com servidor embutido.	API registrada no Eureka e acessada por gateway.
Pegadinha	Não transforma automaticamente a aplicação em microsserviço.	Não significa necessariamente hospedar em AWS, Azure, GCP ou nuvem pública.

4.2 O que significa cloud aqui?

No contexto de Spring Cloud, "cloud" está mais ligado a padrões de cloud-native/distribuídos do que a um provedor específico. Você pode usar conceitos de Spring Cloud em datacenter corporativo, nuvem privada, Kubernetes, ambiente híbrido ou nuvem pública. A prova pode tentar dizer que Spring Cloud é "serviço de nuvem" ou "provedor de infraestrutura"; essa leitura é imprecisa.

5. Service Discovery: descoberta dinâmica de serviços

Service Discovery é o mecanismo pelo qual clientes ou gateways descobrem onde estão as instâncias de um serviço em tempo de execução. Em uma arquitetura estática, alguém configura manualmente URLs como `http://10.1.2.3:8080`. Em uma arquitetura dinâmica, instâncias sobem, descem, escalam e mudam de endereço. O registro de serviços evita acoplamento rígido a endereços físicos.

5.1 Componentes do padrão

Componente	Responsabilidade	Erro comum
Service Registry	Catálogo dinâmico de serviços e instâncias disponíveis.	Confundir com banco de dados comercial.
Service Provider	Serviço que se registra com nome, host, porta, status e metadados.	Achar que apenas serviços públicos se registram.
Service Consumer	Cliente que consulta o registro para localizar instâncias.	Achar que consumidor é sempre navegador.
Health / heartbeat	Sinal periódico de vida ou verificação de disponibilidade.	Achar que heartbeat garante ausência de erro funcional.
Load balancing	Escolha de uma instância entre várias disponíveis.	Confundir descoberta com balanceamento; são conceitos associados, mas distintos.

5.2 Client-side discovery versus server-side discovery

Modelo	Como funciona	Vantagem	Risco / trade-off
Client-side discovery	O cliente consulta o registro e escolhe a instância do serviço.	Menos intermediários no caminho da chamada; controle no cliente.	Cliente precisa conhecer biblioteca/protocolo de descoberta.
Server-side discovery	Cliente chama um endpoint intermediário, como gateway/load balancer, que resolve a instância.	Cliente fica mais simples e desacoplado do mecanismo de descoberta.	Mais responsabilidade e possível gargalo no intermediário.

No ecossistema Spring Cloud Netflix, Eureka é fortemente associado ao client-side discovery: aplicações podem registrar-se e consultar o registro. Já um API Gateway pode combinar rotas com `DiscoveryClient` para resolver dinamicamente destinos.

6. Eureka: registro e descoberta de serviços

Eureka é um componente Netflix OSS integrado pelo Spring Cloud Netflix para Service Discovery. O Eureka Server mantém um registro de instâncias; Eureka Clients registram-se e consultam o registro. A documentação do Spring Cloud Netflix destaca que o service discovery é central em arquiteturas de microsserviços porque configurar manualmente cada cliente é difícil e frágil.

6.1 Fluxo básico com Eureka

1. API processo-service sobe com `spring.application.name=processo-service`.
2. processo-service registra host, porta, health/status e metadados no Eureka Server.
3. API documento-service também se registra.
4. Um gateway ou cliente interno consulta Eureka por processo-service.
5. O cliente obtém uma instância disponível e envia a requisição.
6. Enquanto saudável, a instância envia heartbeats; se falhar por tempo configurado, pode ser removida do registro.

6.2 Exemplo mínimo conceitual

```
# application.yml do serviço cliente Eureka
spring:
  application:
```

```
name: processo-service
server:
  port: 8081
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/
```

O detalhe de prova não é decorar YAML. O que importa: **spring.application.name** tende a virar identidade lógica do serviço; **defaultZone** aponta para o Eureka Server; a instância publica informações para que outros componentes a encontrem.

6.3 O que Eureka não faz

- Não é API Gateway: não é a porta pública de entrada da arquitetura.
- Não é banco de dados transacional: seu registro é operacional e dinâmico.
- Não substitui autenticação, autorização, rate limit ou validação de contrato.
- Não garante que uma chamada será bem-sucedida; apenas ajuda a localizar instâncias registradas e consideradas disponíveis conforme seus critérios.
- Não elimina a necessidade de observabilidade, timeout, retry e circuit breaker.

7. API Gateway: entrada, roteamento e políticas transversais

API Gateway é um componente de borda ou entrada que recebe chamadas de clientes e as encaminha para serviços internos. Em sistemas com múltiplas APIs, ele evita que o cliente conheça diretamente cada serviço, porta, hostname e política técnica. Também concentra preocupações transversais, como autenticação, autorização, CORS, TLS termination, rate limiting, logging, métricas, tracing, transformação de cabeçalho e roteamento por path/host/header.

7.1 Fluxo textual clássico

Angular/mobile -> API Gateway -> serviço de autenticação

-> processo-service
-> documento-service
-> notificacao-service

Cliente externo conhece o gateway.

Gateway conhece rotas e políticas.

Serviços internos permanecem protegidos e evoluem com menor exposição direta.

7.2 Boas responsabilidades do gateway

Responsabilidade adequada	Exemplo
Roteamento	Enviar /processos/** para processo-service e /documentos/** para documento-service.
Política transversal	Validar token, aplicar CORS, registrar logs, medir latência, rate limit.
Adaptação leve	Reescrever path, adicionar/remover cabeçalho técnico, aplicar versão de rota.
Resiliência técnica	Timeout, retry limitado, fallback técnico, circuit breaker quando aplicável.
Integração com discovery	Resolver lb://processo-service por DiscoveryClient/Eureka.

7.3 Más responsabilidades do gateway

- Concentrar regra negocial profunda, como cálculo de prazo, decisão de alçada, validação de domínio ou persistência.
- Virar “monólito de borda”, onde toda mudança negocial exige alterar o gateway.
- Substituir API Management, governança de contrato ou documentação de APIs.
- Fazer autenticação sem validação nos serviços quando a arquitetura exige defesa em profundidade.
- Ser ponto único de falha sem alta disponibilidade, métricas e estratégia de degradação.

Ponto FGV: API Gateway centraliza entrada e políticas técnicas, mas não deve ser tratado como depósito de regra de negócio. A alternativa “quase certa” costuma exagerar sua função.

8. Spring Cloud Gateway e Zuul

O edital cita Zuul expressamente e também cobra API Gateway. Hoje, em projetos Spring mais novos, Spring Cloud Gateway é a solução central do ecossistema Spring para gateway. Zuul é um componente Netflix OSS historicamente usado para roteamento inteligente e filtros. Para prova, você precisa dominar os dois nomes e a distinção funcional com Eureka.

8.1 Spring Cloud Gateway

Spring Cloud Gateway fornece um API Gateway no ecossistema Spring. **A documentação oficial descreve rotas, predicados e filtros. Uma rota combina um identificador, URI de destino, predicados que decidem se a rota se aplica e filtros que modificam requisições/respostas antes ou depois do encaminhamento.**

Conceito	No Spring Cloud Gateway	Exemplo mental
Route	Regra de roteamento para um destino.	id=processos, uri=lb://processo-service
Predicate	Condição para aplicar a rota.	Path=/processos/** ou Host=api.tjsc.local
Filter	Modificação de request/response ou política transversal.	AddRequestHeader, RewritePath, CircuitBreaker
DiscoveryClient locator	Cria rotas baseadas em serviços descobertos.	/processo-service/** -> lb://processo-service
lb://	URI lógica balanceada por integração com discovery/load balancer.	lb://documento-service

spring:

cloud:

gateway:

routes:

- id: processos

uri: lb://processo-service

predicates:

- Path=/processos/**

filters:

- RewritePath=/processos/(?<segment>.*), /\${segment}

8.2 Zuul

Zuul, no contexto Spring Cloud Netflix tradicional, é associado a intelligent routing e gateway/reverse proxy. **Ele intercepta requisições e pode aplicar filtros antes, durante e depois do roteamento.** Em provas, Zuul costuma aparecer como API Gateway ou roteador de borda no ecossistema Netflix OSS, enquanto Eureka aparece como Service Discovery.

8.3 Zuul versus Spring Cloud Gateway

Critério	Zuul	Spring Cloud Gateway
Origem	Netflix OSS, integrado historicamente pelo Spring Cloud Netflix.	Projeto do ecossistema Spring Cloud.
Papel	Roteamento inteligente/gateway/reverse proxy com filtros.	API Gateway com rotas, predicados e filtros.
Situação em prova	Cobrado por constar no edital; tratar como componente de gateway legado.	Solução atual do ecossistema Spring para API Gateway.
Risco de confusão	Dizer que Zuul é registry de serviços.	Dizer que Spring Cloud Gateway é banco de discovery.
Relação com Eureka	Pode usar discovery para rotear por serviceId.	Pode usar DiscoveryClient e lb://service-name.

Memorize: Eureka descobre serviços. Zuul/Spring Cloud Gateway roteiam chamadas e aplicam filtros. Spring Boot cria aplicações. Spring Cloud fornece padrões distribuídos. API Gateway é função arquitetural; Zuul e Spring Cloud Gateway são implementações possíveis dessa função.

9. Service Discovery versus API Gateway versus Load Balancer

Pergunta	Service Discovery	API Gateway	Load Balancer
Pergunta que responde	Onde estão as instâncias de um serviço?	Por onde o cliente entra e para onde a chamada deve ir?	Qual instância receberá a requisição?
Exemplo	Eureka registry.	Zuul ou Spring Cloud Gateway.	Spring Cloud LoadBalancer,

			NGINX, balanceador de nuvem.
Escopo	Catálogo dinâmico de instâncias.	Borda, roteamento e políticas transversais.	Distribuição de tráfego.
Dado principal	Nome lógico, host, porta, status e metadados.	Path, host, header, método, token, destino.	Conjunto de instâncias e algoritmo.
Erro típico	Achar que autentica e roteia tudo sozinho.	Achar que registra instâncias e emite heartbeats.	Achar que resolve governança e contrato de APIs.

10. Arquitetura integrada: Angular + Gateway + Eureka + Spring Boot

Cenário: o TJSC possui uma aplicação Angular para consulta e movimentação de processos administrativos. O front-end chama um endereço único, como <https://api.tjsc.local>. Por trás desse endereço há um API Gateway. Os serviços processo-service, usuario-service e documento-service são aplicações Spring Boot registradas no Eureka. O gateway usa rotas para encaminhar /processos/**, /usuarios/** e /documentos/** aos serviços corretos.

Cliente Angular/mobile

|
| HTTPS + JSON
v

API Gateway (Zuul ou Spring Cloud Gateway)

| consulta rotas e, se configurado, discovery
v

Eureka Server <---- processo-service / usuario-service / documento-service se registram

|
v

Instância escolhida do serviço Spring Boot

|
v

Banco, mensageria ou integrações internas

Nessa arquitetura, o cliente não precisa conhecer a quantidade de instâncias nem seus endereços físicos. O gateway pode aplicar autenticação e políticas técnicas. Eureka permite que o endereço físico dos serviços varie. Spring Boot facilita a construção de cada aplicação. Spring Cloud articula os padrões distribuídos.

11. Trade-offs que a FGV pode explorar

Decisão	Benefício	Custo / risco
Usar API Gateway	Entrada única, políticas transversais, menor exposição interna.	Pode virar gargalo, ponto único de falha ou "monólito de borda".
Usar Service Discovery	Evita URLs fixas e suporta escalabilidade dinâmica.	Exige registry confiável, cache, heartbeat e tratamento de inconsistência temporária.
Usar Spring Boot	Produtividade, convenções, servidor embutido, configuração externa.	Pode ocultar decisões se o time não entende auto-configuração.
Usar Spring Cloud	Padrões distribuídos integrados ao ecossistema Spring.	Aumenta complexidade operacional e dependência de componentes.
Usar Zuul legado	Conhecido em arquiteturas Netflix OSS antigas.	Pode não ser a escolha moderna; prova deve separar legado, função e componente.
Usar gateway com regras demais	Parece centralizar controle.	Aumenta acoplamento e dificulta evolução dos serviços.

12. Pegadinhas FGV

1. Spring Boot não é Spring Cloud: Boot facilita criação/execução de aplicação; Cloud trata padrões distribuídos.
2. Spring Cloud não é provedor de nuvem: é ecossistema de bibliotecas e padrões para sistemas distribuídos.
3. Auto-configuração não significa ausência total de configuração: significa configuração automática condicionada a dependências e propriedades.
4. Eureka não é gateway: ele registra e descobre serviços; não é a entrada pública principal da arquitetura.
5. Zuul não é service registry: ele roteia/proxy/filtros; Eureka registra/descobre.
6. API Gateway não deve concentrar regra de negócio profunda: a borda aplica políticas técnicas e roteamento.

7. Service Discovery não substitui DNS em todos os casos, nem DNS substitui todos os recursos de registry dinâmico.
8. Load balancing e discovery são próximos, mas não idênticos: descobrir instâncias não é necessariamente escolher uma.
9. Gateway pode melhorar governança, mas também adiciona latência e risco de gargalo.
10. A existência de Eureka/Gateway não torna uma arquitetura automaticamente resiliente; timeouts, retries, circuit breakers, métricas e HA ainda importam.

13. Questões autorais - padrão FGV

As questões a seguir são autorais. Elas não reproduzem questões reais; foram construídas para treinar o nível de distinção conceitual esperado para a FGV.

Questão 1. Em uma arquitetura de sistemas distribuídos, uma equipe decide utilizar Spring Boot para construir uma API Java de consulta processual. Considerando o papel do Spring Boot, assinale a afirmativa mais adequada.

- A) Spring Boot substitui a necessidade de projeto arquitetural, pois define automaticamente se a aplicação será monolítica, SOA ou microsserviço.
- B) Spring Boot é, essencialmente, um serviço de descoberta que registra dinamicamente as instâncias da aplicação.
- C) Spring Boot facilita a criação de aplicações Spring executáveis e prontas para produção, por meio de convenções, starters, auto-configuração e configuração externa.
- D) Spring Boot é um API Gateway reativo destinado a aplicar filtros de borda e roteamento por predicados.
- E) Spring Boot é um servidor de mensageria usado para publicar eventos comerciais entre microsserviços.

Questão 2. Em um sistema com vários serviços Spring Boot, cada serviço registra seu nome lógico, endereço, porta e metadados em um componente central. Outros serviços consultam esse componente para localizar instâncias disponíveis. O padrão descrito é:

- A) API Gateway, pois todo componente que concentra nomes de serviços é gateway.
- B) ORM, pois mapeia nomes lógicos para endereços físicos.
- C) Event sourcing, pois registra eventos de subida e descida das instâncias.
- D) Service Discovery, pois permite localizar dinamicamente instâncias de serviços.
- E) MVC, pois separa Model, View e Controller em serviços distintos.

Questão 3. A respeito da relação entre Eureka e Zuul no ecossistema Spring Cloud Netflix tradicional, analise as afirmativas a seguir. I. Eureka está associado ao registro e à descoberta de serviços. II. Zuul está associado a roteamento inteligente, gateway e filtros de requisição. III. Zuul e Eureka são sinônimos, pois ambos executam a mesma função de registry. Está correto o que se afirma em:

- A) I, apenas.
- B) I e II, apenas.
- C) I e III, apenas.
- D) II e III, apenas.
- E) I, II e III.

Questão 4. Um tribunal expõe a clientes web e mobile apenas o endereço `https://api.exemplo.jus.br`. Internamente, chamadas para `/processos/**` são encaminhadas ao serviço de processos, `/documentos/**` ao serviço documental e `/usuarios/**` ao serviço de usuários. O componente de entrada também aplica autenticação, CORS, logs e métricas. A descrição corresponde mais diretamente a:

- A) um repositório JPA, pois centraliza o acesso aos dados.
- B) um barramento de eventos, pois distribui chamadas HTTP síncronas como mensagens assíncronas.
- C) um API Gateway, pois centraliza entrada, roteamento e preocupações transversais.
- D) um Eureka Server, pois todo roteamento por path é função nativa do registry.
- E) um objeto de valor de DDD, pois encapsula regras de identidade dos serviços.

Questão 5. Em relação à auto-configuração do Spring Boot, assinale a alternativa correta.

- A) A auto-configuração é invasiva e impede que o desenvolvedor substitua beans ou configurações específicas.
- B) A auto-configuração tenta configurar a aplicação com base em dependências presentes e propriedades, mas pode ser ajustada ou substituída.
- C) A auto-configuração é um mecanismo exclusivo do Spring Cloud Gateway para criação automática de rotas.
- D) A auto-configuração dispensa a separação entre camadas, pois coloca controllers, services e repositories em um único bean.

E) A auto-configuração elimina a necessidade de variáveis de ambiente e arquivos YAML em qualquer cenário.

Questão 6. Considere o seguinte fluxo textual: Cliente Angular -> Gateway -> lb://processo-service -> instância do processo-service. O prefixo lb://, nesse contexto, sugere principalmente:

- A) uma rota lógica balanceada, geralmente resolvida com apoio de discovery/load balancer.
- B) um protocolo criptográfico substituto do TLS.
- C) um formato JSON usado para serializar a resposta.
- D) uma anotação JPA para relacionamento lazy/batch.
- E) um canal de mensageria do RabbitMQ.

Questão 7. Sobre Spring Cloud, assinale a afirmativa mais adequada para prova.

- A) É um provedor de nuvem pública que hospeda aplicações Spring Boot sem infraestrutura própria.
- B) É um substituto do Java Virtual Machine para execução nativa obrigatória de aplicações web.
- C) É um conjunto de projetos que auxilia na implementação de padrões comuns em sistemas distribuídos, como descoberta, configuração, roteamento e resiliência.
- D) É uma biblioteca de persistência objeto-relacional equivalente ao Hibernate.
- E) É uma linguagem de programação criada para substituir Java em microsserviços.

Questão 8. Uma equipe colocou no API Gateway regras de cálculo de prazo processual, validações de domínio, decisões de alçada e persistência de auditoria negocial. Sob a ótica arquitetural, essa decisão é problemática porque:

- A) o gateway deve ser utilizado apenas para persistência de dados, e não para receber requisições HTTP.
- B) service discovery só funciona quando o gateway contém todas as regras de negócio.
- C) Eureka exige que o gateway concentre regras negociais para registrar serviços.
- D) Spring Boot não permite que gateways executem filtros.
- E) o gateway passa a concentrar responsabilidade negocial profunda, aumentando acoplamento e risco de um monólito de borda.

Questão 9. Associe os componentes às funções. 1. Spring Boot 2. Eureka 3. Zuul 4. Spring Cloud Gateway () Registro e descoberta de instâncias de serviços. () Construção simplificada de aplicações Spring executáveis. () Gateway/roteamento inteligente legado no ecossistema Netflix OSS. () API Gateway do ecossistema Spring baseado em rotas, predicados e filtros. A sequência correta é:

- A) 2 - 1 - 3 - 4.
- B) 1 - 2 - 4 - 3.
- C) 3 - 1 - 2 - 4.
- D) 2 - 3 - 1 - 4.
- E) 4 - 2 - 3 - 1.

Questão 10. Em uma aplicação Spring Boot, a equipe deseja manter o mesmo código para desenvolvimento, homologação e produção, alterando URLs, credenciais, portas e flags por ambiente. O recurso conceitualmente mais adequado é:

- A) Object Relational Mapping.
- B) Model View Controller.
- C) compilação de templates server-side.
- D) configuração externa por arquivos, variáveis de ambiente, perfis e argumentos.
- E) registro obrigatório de todas as propriedades no Eureka Server.

Questão 11. Sobre Service Discovery, assinale a alternativa incorreta.

- A) Ajuda a evitar acoplamento rígido a endereços físicos de instâncias.
- B) Substitui integralmente autenticação, autorização, monitoramento e tratamento de falhas.
- C) Pode usar um registro de serviços consultado por clientes ou gateways.
- D) É útil quando instâncias sobem, descem ou escalam dinamicamente.
- E) Pode trabalhar junto com mecanismos de balanceamento de carga.

Questão 12. Analise as afirmativas. I. Um API Gateway pode aplicar políticas transversais, como autenticação, logging e rate limiting. II. Um API Gateway deve, como regra, conter toda a lógica de domínio para impedir que serviços internos tenham regras repetidas. III. Um API Gateway pode rotear chamadas com base em path, host, headers ou outros critérios de requisição. Está correto o que se afirma em:

- A) I, apenas.
- B) II, apenas.
- C) I e III, apenas.
- D) II e III, apenas.
- E) I, II e III.

Questão 13. Uma instância de serviço-documentos envia periodicamente heartbeats ao Eureka. Depois de falhas sucessivas, ela pode deixar de ser considerada disponível no registro. Esse comportamento está relacionado:

- A) à geração automática de DTOs pelo Spring Boot.
- B) ao mapeamento de entidades JPA em tabelas relacionais.
- C) à manutenção operacional do registro de serviços com base em sinais de vida/saúde.
- D) à conversão obrigatória de HTTP para AMQP.
- E) à substituição do API Gateway por DNS estático.

Questão 14. No Spring Cloud Gateway, os conceitos de Route, Predicate e Filter correspondem, respectivamente, a:

- A) regra de encaminhamento; condição de correspondência da requisição; modificação ou ação aplicada à requisição/resposta.
- B) entidade de domínio; objeto de valor; repositório de persistência.
- C) cliente Eureka; servidor Eureka; heartbeat.
- D) controlador MVC; view resolver; model attribute.
- E) fila; exchange; routing key.

Questão 15. Em uma arquitetura com clientes externos, gateway, Eureka e serviços Spring Boot, assinale a leitura tecnicamente mais adequada.

- A) O cliente externo deve consultar diretamente o Eureka para descobrir serviços internos, pois isso sempre aumenta segurança.
- B) O Eureka deve processar a autenticação de usuários finais e aplicar CORS.
- C) O Spring Boot deve ser implantado dentro do banco de dados para habilitar service discovery.
- D) O gateway pode ser a entrada conhecida pelos clientes, enquanto serviços internos registram-se no Eureka para descoberta dinâmica.
- E) O uso de Eureka elimina a necessidade de contratos de API e versionamento.

Questão 16. Em relação a Zuul, no contexto cobrado pelo edital, é mais correto afirmar que:

- A) é uma especificação de persistência Java equivalente à JPA.
- B) é um componente associado a gateway/reverse proxy/roteamento inteligente no ecossistema Netflix OSS, historicamente integrado ao Spring Cloud Netflix.
- C) é um formato de payload concorrente do JSON.
- D) é o servidor de service discovery que substitui Eureka.
- E) é uma técnica de modelagem de domínio equivalente a agregado DDD.

Questão 17. Um candidato afirma: "Como a aplicação usa @SpringBootApplication, então ela já é automaticamente um microsserviço registrado no Eureka e exposto por gateway". A afirmação é:

- A) correta, porque @SpringBootApplication sempre cria API Gateway e registry.
- B) correta apenas quando a aplicação usa banco relacional.
- C) incorreta, pois Spring Boot facilita a aplicação, mas registro no Eureka, exposição por gateway e arquitetura de microsserviços dependem de componentes e decisões adicionais.
- D) incorreta apenas porque Eureka não pode ser usado com Spring Boot.
- E) correta, desde que a aplicação retorne JSON.

Questão 18. Considere as assertivas sobre gateway e service discovery. I. Service Discovery responde à necessidade de localizar instâncias de serviços. II. API Gateway responde à necessidade de entrada, roteamento e políticas transversais. III. Load balancing responde à escolha de instância entre alternativas disponíveis. A classificação está:

- A) errada apenas em I.
- B) correta em I, II e III.
- C) errada apenas em II.

D) errada apenas em III.

E) errada em I, II e III.

Questão 19. Uma equipe de integração deseja expor APIs internas do TJSC para múltiplos clientes sem que cada cliente conheça endereços físicos dos serviços, portas e quantidade de instâncias. A solução arquitetural mais coerente, considerando os itens da aula, combina:

A) API Gateway como ponto de entrada e Service Discovery para localização dinâmica de serviços internos.

B) JPA como roteador de borda e Hibernate como registry.

C) Acesso direto dos clientes ao banco e eliminação do backend.

D) Apenas DNS estático, pois registry dinâmico e gateway são sempre antipatronos.

E) MVC como substituto de todos os protocolos de integração.

Questão 20. Em uma prova, aparece a afirmação: “Spring Cloud Gateway pode criar rotas com base em serviços registrados em uma implementação de DiscoveryClient, como Eureka, utilizando padrões como /serviceld/** e lb://service-name”. À luz da documentação e do uso arquitetural, a afirmação é:

A) falsa, pois gateway nunca pode consultar discovery.

B) falsa, pois Eureka só armazena entidades JPA.

C) falsa, pois lb:// é sintaxe exclusiva de RabbitMQ.

D) verdadeira, pois o gateway pode integrar-se ao DiscoveryClient para criar/usar rotas baseadas em serviços registrados.

E) verdadeira apenas se todos os serviços estiverem em um único monólito.

14. Gabarito resumido

1-5	6-10	11-15	16-20	
1C, 2D, 3B, 4C, 5B	6A, 7C, 8E, 9A, 10D	11B, 12C, 13C, 14A, 15D	16B, 17C, 18B, 19A, 20D	

Gabarito em sequência: 1C - 2D - 3B - 4C - 5B - 6A - 7C - 8E - 9A - 10D - 11B - 12C - 13C - 14A - 15D - 16B - 17C - 18B - 19A - 20D

Questão 1 - Gabarito: C

A correta é C. Spring Boot reduz o esforço de configuração e empacotamento de aplicações Spring, especialmente por starters, auto-configuração e configuração externa. A alternativa A exagera: arquitetura não nasce automaticamente do framework. B descreve Eureka, não Boot. D descreve gateway, especialmente Spring Cloud Gateway. E descreve broker/mensageria, tema de outra aula.

Questão 2 - Gabarito: D

A correta é D. O cenário descreve exatamente Service Discovery: registro e localização dinâmica de instâncias. A é tentadora porque gateways podem consultar discovery, mas gateway é entrada/roteamento. B confunde mapeamento de nomes com ORM. C usa linguagem de eventos, mas o objetivo não é reconstruir estado de domínio por eventos. E traz MVC, que trata separação Model-View-Controller, não localização de serviços.

Questão 3 - Gabarito: B

A correta é B. I está certa: Eureka é service registry/discovery. II está certa: Zuul é historicamente associado a roteamento inteligente/gateway/filtros. III está errada porque Zuul e Eureka têm funções diferentes. A ignora II. C mantém uma assertiva falsa. D exclui I sem razão. E aceita a equivalência indevida entre Zuul e Eureka.

Questão 4 - Gabarito: C

A correta é C. O componente descrito centraliza entrada, roteia por path e aplica políticas transversais: típico API Gateway. A fala de persistência. B confunde chamada HTTP síncrona com eventos assíncronos. D é o distrator forte: Eureka descobre serviços, mas não é o roteador público por path. E desloca o tema para DDD.

Questão 5 - Gabarito: B

A correta é B. A auto-configuração do Spring Boot é condicional e pode ser substituída pelo desenvolvedor. A erra ao dizer que é invasiva. C confunde auto-configuração com gateway. D transforma convenção de framework em antipadrão de camadas. E exagera: configuração externa continua relevante e frequentemente essencial.

Questão 6 - Gabarito: A

A correta é A. Em Spring Cloud Gateway, lb://service-name indica destino lógico balanceado, geralmente com integração a discovery/load balancer. B confunde com TLS. C confunde com payload. D puxa persistência. E puxa mensageria, pois RabbitMQ trabalha com filas/exchanges/routing keys, não com lb:// nesse contexto.

Questão 7 - Gabarito: C

A correta é C. Spring Cloud reúne projetos para padrões de sistemas distribuídos. A confunde "Cloud" com provedor de nuvem. B inventa substituição da JVM. D descreve ORM, como JPA/Hibernate. E inventa linguagem de programação.

Questão 8 - Gabarito: E

A correta é E. O gateway pode aplicar políticas técnicas, mas regra negocial profunda deve ficar nos serviços/domínio. A inverte o papel do gateway. B cria dependência falsa entre discovery e regra de negócio no gateway. C atribui exigência inexistente ao Eureka. D é falsa: gateways usam filtros, mas filtros não devem virar domínio inteiro.

Questão 9 - Gabarito: A

A correta é A: Eureka = 2, Spring Boot = 1, Zuul = 3, Spring Cloud Gateway = 4. B troca discovery com Boot e gateway com Zuul/Gateway. C coloca Zuul como registry. D coloca Zuul no papel de Boot. E inicia com Spring Cloud Gateway como registry, erro clássico de escopo.

Questão 10 - Gabarito: D

A correta é D. Configuração externa permite variar propriedades por ambiente sem alterar código. A é ORM. B é padrão de apresentação. C pode existir em web server-side, mas não resolve configuração de ambiente. E é erro: Eureka registra serviços; não é cofre universal de propriedades da aplicação.

Questão 11 - Gabarito: B

A incorreta é B. Service Discovery não substitui autenticação, autorização, monitoramento nem tratamento de falhas. A, C, D e E são corretas: ele reduz acoplamento a endereços físicos, usa registro, é útil em ambiente dinâmico e pode atuar com balanceamento. A pegadinha está em transformar uma capacidade útil em solução total.

Questão 12 - Gabarito: C

A correta é C. I está certa: gateway é local comum para políticas transversais. II está errada: regra de domínio profunda no gateway aumenta acoplamento. III está certa: roteamento por path/host/header é típico de gateways. A ignora III. B aceita apenas a errada. D combina II falsa com III verdadeira. E aceita tudo indevidamente.

Questão 13 - Gabarito: C

A correta é C. Heartbeats e saúde estão ligados à manutenção operacional do service registry. A descreve geração de DTO, que não é o caso. B fala de persistência. D inventa conversão HTTP-AMQP. E força oposição com DNS estático.

Questão 14 - Gabarito: A

A correta é A. Route define a regra/destino; Predicate decide se a requisição casa com a rota; Filter modifica ou aplica ações antes/depois. B é DDD/persistência. C é Eureka. D é Spring MVC. E é mensageria RabbitMQ.

Questão 15 - Gabarito: D

A correta é D. O gateway pode ser a entrada externa e os serviços podem registrar-se no Eureka para descoberta interna. A é perigosa: expor discovery diretamente a cliente externo pode ferir segurança e acoplamento. B atribui autenticação final e CORS ao Eureka. C é nonsense técnico. E exagera: discovery não elimina contratos/versionamento.

Questão 16 - Gabarito: B

A correta é B. Zuul é associado a gateway/reverse proxy/roteamento inteligente no Netflix OSS. A descreve JPA. C descreve formato de dados. D troca Zuul com Eureka. E troca arquitetura de integração com DDD.

Questão 17 - Gabarito: C

A correta é C. @SpringBootApplication indica aplicação Spring Boot, mas não implica automaticamente microsserviço, Eureka ou gateway. A junta funções de componentes distintos. B condiciona indevidamente a banco relacional. D nega compatibilidade entre Eureka e Spring Boot. E confunde JSON com arquitetura.

Questão 18 - Gabarito: B

A correta é B. As três assertivas classificam corretamente os papéis: discovery localiza instâncias; gateway controla entrada/roteamento/políticas; load balancing escolhe instância. As demais alternativas tentam negar uma classificação correta sem fundamento.

Questão 19 - Gabarito: A

A correta é A. Para esconder endereços físicos dos clientes e lidar com instâncias dinâmicas, combina-se gateway como entrada com discovery para serviços internos. B troca funções com JPA/Hibernate. C viola camadas e segurança. D absolutiza DNS estático e chama padrões úteis de antipatronos. E desloca MVC para integração, o que não se sustenta.

Questão 20 - Gabarito: D

A correta é D. Spring Cloud Gateway pode integrar-se com DiscoveryClient e usar rotas baseadas em serviceId/lb://service-name. A nega uma capacidade documentada. B confunde Eureka com persistência. C confunde lb:// com mensageria. E limita indevidamente o recurso a monólito único, quando ele faz sentido justamente em serviços distribuídos.

15. Checklist final da aula

- Diferenciar Spring Boot de Spring Cloud.
- Diferenciar Eureka de Zuul e Spring Cloud Gateway.
- Saber que Service Discovery localiza instâncias, mas não substitui segurança, resiliência e observabilidade.
- Saber que API Gateway roteia e aplica políticas transversais, mas não deve concentrar domínio profundo.
- Entender rotas, predicados, filtros, lb:// e DiscoveryClient em nível conceitual de prova.
- Perceber trade-offs: produtividade versus complexidade, centralização versus gargalo, discovery dinâmico versus consistência/cache.

16. Referências técnicas consultadas

- Edital TJSC 2026 - Analista de Sistemas - FGV: <https://conhecimento.fgv.br/concursos/tjscservidor26>
- Spring Boot Reference - Overview: <https://docs.spring.io/spring-boot/index.html>
- Spring Boot Reference - Auto-configuration: <https://docs.spring.io/spring-boot/reference/using/auto-configuration.html>
- Spring Boot Reference - Externalized Configuration: <https://docs.spring.io/spring-boot/reference/features/external-config.html>
- Spring Cloud Netflix Reference - Eureka and legacy Netflix OSS components: <https://docs.spring.io/spring-cloud-netflix/docs/current/reference/html/>
- Spring Cloud Netflix 2.2.x Reference - Eureka, Zuul, Ribbon, Hystrix: <https://docs.spring.io/spring-cloud-netflix/docs/2.2.x/reference/html/>
- Spring Cloud Gateway Reference: <https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/>
- Spring Cloud Gateway - DiscoveryClient Route Definition Locator: <https://docs.spring.io/spring-cloud-gateway/reference/spring-cloud-gateway-server-webflux/the-discoveryclient-route-definition-locator.html>
- Spring Cloud Gateway - GatewayFilter Factories: <https://docs.spring.io/spring-cloud-gateway/reference/spring-cloud-gateway-server-webflux/gatewayfilter-factories.html>